

# Lisno

# Full-stack safeguards review

A code-grounded review of identity, client data, deployment, recovery, and operational guardrails across the web app, API, mobile app, and OCR worker.

| P0 / release blocker | P1 / fix before rollout | P2 / hardening cycle |
|----------------------|-------------------------|----------------------|
| 1 confirmed          | 4 priority gaps         | 8 targeted controls  |

### Executive decision

**Do not enable public production Client signup and project claiming yet.** The current endpoint creates an active account, links projects by submitted email, and issues a JWT before proving email ownership. The repository already labels this a production blocker. The next release gate should also require a two-tab session isolation test, denial-aware cache retirement, and a passing automated pipeline.

Operationally, the tracked Render blueprint relies on a single upload disk and a static health response. Backup and restore capability, production headers, external monitoring, and branch protections may exist outside this repository; they must be verified before being counted as safeguards.

**Assessment type:** static source review plus focused local type checks and tests. This is not a penetration test or live deployment audit.

# 01 System and trust boundaries

The platform holds private client plans, approval history, financial ledgers, credentials, uploads, and worker job tokens. Its main boundaries are browser and mobile sessions to the API, API to MongoDB and file storage, and the OCR worker to internal leased jobs.

| Surface                    | Role in the system                                                             | Trust boundary                                            |
|----------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------|
| Web / React + Vite         | Staff and client workspaces; TanStack Query cache; bearer-session transport    | Browser JavaScript, local storage, and API                |
| Mobile / Expo              | Native staff workflows; environment-specific API; SecureStore token vault      | Device credential store and API                           |
| API / Express + Mongoose   | Authentication, operation permissions, workflow, finance, files, mail          | Public routes, role authorization, MongoDB, local uploads |
| OCR / Python               | Claims leased extraction jobs; processes uploaded PDFs and images              | Worker token, claim token, file parsing, internal API     |
| Render + external services | API, static web, OCR worker, upload disk; Mongo and mail configured externally | Runtime secrets, persistence, deployment checks           |

## Safeguards already visible

**Backend:** Zod input and environment validation; operation-based authorization; JWT algorithm pinning and session-version checks; transactional identity changes; upload size and type validation; opaque storage references; demo identities restricted to local development.

**Web:** validated authorization snapshots; permission routes; fragment token scrubbing before router mount; keyboard/focus foundations; a dashboard policy that retires data after denied refetches.

**Mobile and OCR:** SecureStore token isolation by environment, production HTTPS checks, abortable HTTP requests, leased OCR jobs, heartbeat/retry behavior, and page/pixel/output limits.

**Evidence:** backend/src/middleware/authorization.ts; backend/src/services/auth.service.ts:290-295; frontend/src/auth/authorization.ts; mobile/src/core/session/tokenVault.ts; mobile/app.config.ts:67-80; ocr-worker/src/lisno\_ocr/contracts.py:132-170.

## Priority scale and certainty

**P0** blocks external production use. **P1** should close before broad rollout. **P2** belongs in the next hardening cycle. **P3** is a measured improvement. A source-confirmed behavior is described as *observed*; missing repository configuration is a *repo gap*; live controls outside the repo remain *unverified*.

## 02 Identity and data exposure

These controls protect the boundary between one person, one client account, and one set of authorized projects.

P0

F01 Email claim precedes ownership proof

**Observed.** The public client-signup route creates an active Client, links every unclaimed project with the submitted normalized email, and issues a JWT in the same flow. README.md already calls this a production blocker.

**Risk.** A party who knows a prospective client's email can claim that client's unlinked projects. **Guardrail.** Disable production signup/claim until a single-use, expiring email challenge is completed. Create a pending identity, then atomically link projects and issue a session only after verification. Test takeover attempts with two unequal client identities and projects.

Evidence: backend/src/routes/auth.ts:79-90; backend/src/services/auth.service.ts:200-287; README.md:81-83

P1

F02 Browser sessions are not synchronized across tabs

**Observed.** The web token is in localStorage and read for each request. Logout and TanStack cache clearing operate only in the initiating tab; AuthProvider listens for a same-tab custom event, with no storage or BroadcastChannel change handler.

**Risk.** Another tab can keep private cached UI after logout. If a different user signs in elsewhere, its next request can use the new token while still displaying the prior identity. **Guardrail.** Coordinate token changes across tabs, abort active requests, clear private caches, and revalidate identity before rendering. Add two-tab logout and account-switch tests.

Evidence: frontend/src/api/client.ts:54-63,157-169; frontend/src/auth/AuthProvider.tsx:94-119,303-336

P1

F03 Denied refetch can leave private details on screen

**Observed.** ClientProject renders cached project and approved-document data whenever query data exists, even after a later 403/404. Finance ledger rendering also accepts cached data after an error. ClientDashboard already uses a denial-aware retirement policy.

**Risk.** A revocation can leave previously authorized private details visible in an open tab after the server has denied the new read. **Guardrail.** Use one denial-aware query policy: on 401/403/404, retire affected data and close previews; permit explicit stale display only for transient failures. Add project-detail and finance revocation regression tests.

Evidence: frontend/src/features/client/ClientProject.tsx:18-42;  
frontend/src/features/finance/ProjectFinancePanel.tsx:153-179;  
frontend/src/features/client/ClientDashboard.tsx:22-36

### 03 Release and recovery gates

The repository can build deployable services, but the tracked release and recovery controls are incomplete.

P1

F04 No enforced CI gate in the tracked repository

**Observed.** No tracked CI workflow or dependency automation configuration was found. The Render blueprint runs `npm ci` and build for the Node workspaces; it does not run tests, mobile/OCR checks, security scans, or a release smoke suite.

**Risk.** A deployable revision can pass compilation while failing authorization, finance, or workflow regressions. External branch protections were not visible in this review. **Guardrail.** Require a passing pipeline for backend, frontend, mobile, and OCR; add focused contract/permission/finance regressions, secret and dependency scanning, build artifact checks, and a manual approval boundary for production deployment.

Evidence: `render.yaml`:13-15,56,87; `README.md`:304-314; no tracked `.github/.gitlab-ci/CircleCI` config

P1

F05 Coordinated backup and restore is unverified

**Observed.** The Render blueprint stores uploaded originals and derived files on one 10 GB persistent disk. MongoDB is external. No tracked runbook or script for coordinated database-plus-file backup and restoration was found.

**Risk.** A disk loss, accidental deletion, or mismatched database/file restore can make approved documents and audit-linked artifacts unavailable. **Guardrail.** Verify provider backups, define RPO/RTO and retention, back up MongoDB and upload objects as one recoverable set, and perform a documented restore drill with authorization and artifact integrity checks.

Evidence: `render.yaml`:16-19,32-34; `backend/README.md`:26-29; `backend/src/server.ts`:138-139

#### Production evidence required outside the repo

| Control                      | Evidence to collect before treating it as complete                             |
|------------------------------|--------------------------------------------------------------------------------|
| Branch and deploy protection | Required checks, reviewers, protected environment, rollback owner              |
| Data recovery                | Recent backup inventory, retention, encryption, restore date and measured RTO  |
| Runtime security             | Actual response headers, secret manager policy, Mongo roles, TLS configuration |
| Operations                   | Alert routes, on-call ownership, incident and data-retention runbooks          |

## 04 API and operational resilience

P2

F06 Throttles depend on one process and proxy address

**Observed.** Authentication, invitation, and reset limiters use process-local Maps. The auth key uses request.ip/socket address, while no explicit trusted-proxy configuration appears in app setup.

**Risk.** Counters reset or diverge across instances; a proxy can cause unrelated users to share a bucket or misrepresent source addresses. **Guardrail.** Use an edge or shared-store limiter with trusted proxy handling and per-account/recipient controls. Validate source-IP semantics in the deployed topology.

Evidence: backend/src/middleware/auth-rate-limit.ts:30-62; invitation-rate-limit.ts:51-90; password-reset-rate-limit.ts:41-81; backend/src/app.ts:452

P2

F07 Reset issuance can be lost after acknowledgement

**Observed.** A password-reset request schedules issuance via queueMicrotask and immediately returns an accepted result; the durable reset record is created later inside that task.

**Risk.** A process crash or restart between acknowledgement and issuance loses the requested email with no queued recovery item. **Guardrail.** Persist a reset job/outbox before acknowledgement, then lease/retry idempotently. Reuse the notification email dispatcher's durable pattern while preserving the generic public response.

Evidence: backend/src/services/password-reset.service.ts:103-109,142-161; backend/src/services/notification-email-dispatcher.ts:21-40

P2

F08 Health and maintenance failures are silent

**Observed.** The health route always returns 200 and is used by Render. The scheduled reconciliation/cleanup loop discards all Promise.allSettled rejections.

**Risk.** Routing can continue to a process with lost dependencies, while cleanup or receipt backlogs grow without a signal. **Guardrail.** Separate liveness from readiness; check Mongo and critical storage readiness. Emit redacted failure metrics, backlog age, and alerts for each maintenance job without stopping idempotent retries.

Evidence: backend/src/routes/health.ts:5-7; render.yaml:15; backend/src/server.ts:233-259

# 05 Session, browser, and telemetry controls

P2

F09 Browser bearer persistence exceeds the UI promise

**Observed.** The web bearer token is stored in localStorage. The visible Remember me checkbox changes local state but its value is not passed to login; all successful logins persist identically.

**Risk.** A same-origin script injection could read a persistent token; users who uncheck the control still have a persistent session. This review found no proof of an existing XSS exploit. **Guardrail.** Prefer an HttpOnly, Secure, SameSite session or BFF design with CSRF protection. Meanwhile, remove or implement the checkbox honestly, shorten session lifetime where feasible, and enforce CSP at the web origin.

Evidence: frontend/src/api/client.ts:54-63; frontend/src/auth/LoginPage.tsx:80, 221-249, 442-449; frontend/src/auth/AuthProvider.tsx:209-212

P2

F10 Central error and security telemetry is incomplete

**Observed.** Unexpected API exceptions become safe generic 500 responses without central reporting. Login success/failure is not captured as a durable security event. No production web error boundary or capture hook was found.

**Risk.** Operators can miss attack patterns, crash clusters, and user-visible render failures. **Guardrail.** Add request/interaction IDs; structured, redacted API and browser error capture; auth failure/rate-limit events; and alerts tied to counts and latency. Exclude passwords, bearer tokens, links, document contents, and personal fields.

Evidence: backend/src/middleware/errors.ts:26-50; backend/src/routes/auth.ts:52-76; frontend/src/main.tsx:25-29; frontend/src/App.tsx:4-9

P2

F11 Response-header baseline is not declared end to end

**Observed.** The API has no central security-header middleware, and the tracked static web deployment has no response-header rules. Some routes set no-store/nosniff individually, but deployed edge headers were not inspected.

**Risk.** Browser policy and cache protections may differ by route or hosting configuration, especially for authenticated data and downloadable files. **Guardrail.** Define and verify CSP, frame-ancestors, HSTS at HTTPS edge, nosniff, referrer policy, and sensitive-response Cache-Control. Test actual web/API/download responses; tune CSP to required assets.

Evidence: backend/src/app.ts:452-469, 638-640; render.yaml:83-97; backend/src/routes/api-docs.ts:72-97

## 06 Cross-platform guardrails

### P2 F12 Permission snapshots are not refreshed mid-session

**Observed.** The web authorization snapshot is loaded on login/restore, and default query behavior disables focus refetch. A grant or revocation has no common freshness path in the UI.

**Risk.** Menus and action controls can remain stale until reload. The backend still authorizes requests, but cached protected views need a revocation response. **Guardrail.** Refresh policy on focus/reconnect or version change; retire affected query data on reduced permissions. Test both a grant and a revoke during an active session.

Evidence: `frontend/src/auth/AuthProvider.tsx:127-168,192-247`; `frontend/src/app/providers.tsx:13-17`

### P2 F13 Worker builds are not reproducible

**Observed.** The OCR Python project permits broad package ranges, including PaddleOCR/PaddlePaddle, and the Render worker build upgrades pip before installing floating model dependencies.

**Risk.** A rebuild can resolve a different inference/runtime stack without a code change, changing OCR output or availability. **Guardrail.** Commit a tested Python lock/constraints set, build from fixed versions, record model versions/checksums, and run non-model plus sample model tests before promoting worker images.

Evidence: `ocr-worker/pyproject.toml:10-22`; `render.yaml:52-57`

### P3 F14 Frontend recovery and request contracts need bounds

**Observed.** The shared JSON client has no default request timeout or critical-response runtime validation; most role routes import eagerly. The existing local build has an approximately 2.0 MiB main JS asset plus 919 KiB CSS.

**Risk.** A hung request can leave controls busy; malformed 2xx responses can fail late; initial transfer may be expensive on mobile. **Guardrail.** Add operation-aware deadlines and abort, validate key auth/finance/approval envelopes, add a root recovery boundary, split large role routes, and measure real first-load transfer and interaction.

Evidence: `frontend/src/api/client.ts:157-175`; `frontend/src/app/router.tsx:21-69`;  
`frontend/dist/assets/index-D4pywg8e.js`

### P3 F15 Repository and production config hygiene

**Observed.** Generated browser/Expo artifacts and demo-login material are tracked. Production environment parsing also keeps localhost Mongo/CORS defaults; a fresh blueprint does not declare mail delivery variables.

**Risk.** Noise and demo material can be republished accidentally, and a deployment can start with unintended defaults or unavailable invitation/reset mail. External Render settings may already supply them. **Guardrail.** Review tracked artifacts and retention before removal; enforce explicit production Mongo/CORS/mail readiness; add a deployment smoke check for invitation/reset delivery. Do not treat published demo fixtures as production credentials.

Evidence: `.gitignore:1-16`; `backend/src/config/env.ts:61-83`; `render.yaml:20-49`

# 07 Safeguard coverage matrix

This matrix distinguishes working controls from partial controls and external facts that the repository cannot prove.

| Area                       | Visible in code                                                     | Next proof or control                                                            |
|----------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Identity and authorization | Operation registry, JWT/session version, Zod; signup ownership gap  | Verified email before claim; cross-role takeover and revocation tests            |
| Client data isolation      | Server permission checks; partial denied-cache retirement           | Two-tab, 403/404, mid-session revocation matrix                                  |
| Workflow and finance       | Transactions, audit/history, paise-based finance rules              | Automated asymmetric project reconciliation and approval race gates              |
| Uploads and OCR            | Type/size checks, opaque references, leased jobs, extraction limits | Storage restore; worker stream cap; malicious file test set                      |
| Mobile                     | SecureStore, HTTPS production check, abort/stale-session controls   | Release signing, device matrix, offline/restore exercise                         |
| Resilience                 | Some durable job patterns; static health; microtask reset           | Readiness, durable reset queue, backlog alerts, recovery drill                   |
| Delivery                   | Lockfiles for npm; Render build scripts                             | Required CI checks, pinned OCR stack, staged rollback                            |
| Privacy and governance     | Private data and audit trails exist; external policy unknown        | Owner-approved retention, access review, deletion/export and incident procedures |

## Scope boundary

This review sampled high-risk code paths and configuration across 2,426 tracked files. It did not prove that every route, role, transaction, or upload decoder is safe. It did not inspect live Render/Atlas settings, customer data, traffic, backup contents, signing keys, or browser response headers. Security headers and backups may be present outside this repo; record evidence before marking those controls complete.

This assessment made no changes in the Lisno repository. An Expo development log was modified at the start; an unrelated untracked design spec appeared during the review. Both were left untouched.

# 08 Remediation sequence and acceptance

A practical order for engineering and operations, with observable exit criteria rather than feature-completion claims.

| Window          | Work package                                                                            | Exit criteria                                                                                  |
|-----------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Immediate       | F01: production signup gate and verified email design                                   | No project link or session before verified ownership; takeover tests pass                      |
| Before rollout  | F02-F05: cross-tab isolation, denied-cache retirement, CI gate, recovery proof          | Two-tab and 403 tests pass; required checks block deploy; file+Mongo restore succeeds          |
| Next cycle      | F06-F11: shared throttles, durable reset, readiness, telemetry, session/header baseline | Proxy test; reset survives restart; readiness fails on Mongo loss; alerts and headers verified |
| Following cycle | F12-F15: policy freshness, OCR lock, client bounds/performance, repo hygiene            | Grant/revoke tests; reproducible worker build; client perf budget; clean artifact policy       |

## Validation run for this assessment

**Passed:** backend TypeScript no-emit typecheck; frontend app and Vite config no-emit typechecks; mobile TypeScript no-emit typecheck; backend auth/password-reset focused tests (120); frontend auth/client focused tests (70); OCR non-model tests (328 passed, 1 skipped, 1 deselected).

**Observed warning:** the frontend LoginPage test run printed MSW unmatched-request warnings for ancillary endpoints while all selected tests passed. The report does not infer a production network failure from those test warnings.

**Not run:** full JS suites/builds, replica-set integration suite, live browser/mobile interaction, dependency vulnerability scan, penetration test, deployment header inspection, backup restore, and production mail smoke test.

## Independent references used for recommendations

- R1 OWASP Email Validation and Verification Cheat Sheet
- R2 OWASP Session Management Cheat Sheet
- R3 OWASP Authorization Cheat Sheet
- R4 OWASP HTTP Headers Cheat Sheet
- R5 OWASP Logging Cheat Sheet
- R6 OWASP File Upload Cheat Sheet
- R7 NIST Secure Software Development Framework, SP 800-218